DevOps in One Shot

GitHub : Git & GitHub has steadily risen from being just a preferred skill to a must-have skill for multiple job roles today. In this article, I will talk about the Top 30 Git Commands that you will be using frequently while you are working with Git.

Essential GitHub Commands Every DevOps Engineer Should Know

1. **git init -**Initializes a new Git repository in the current directory.
2. **git clone [url]-** Clones a repository into a new directory.
3. **git add [file] -**Adds a file or changes in a file to the staging area.
4. **git commit -m "[message]" -**Records changes to the repository with a descriptive message.
5. **git push -**Uploads local repository content to a remote repository.
6. **git pull -** Fetches changes from the remote repository and merges them into the local branch.
7. **git status -** Displays the status of the working directory and staging area.
8. **git branch -** Lists all local branches in the current repository.
9. **git checkout [branch] -** Switches to the specified branch.
10. **git merge [branch] -** Merges the specified branch's history into the current branch.
11. **git remote -v -** Lists the remote repositories along with their URLs.
12. **git log -** Displays commit logs.
13. **git reset [file] -** Unstages the file, but preserves its contents.
14. **git rm [file] -** Deletes the file from the working directory and stages the deletion.
15. **git stash -** Temporarily shelves (or stashes) changes that haven't been committed.
16. **git tag [tagname] -** Creates a lightweight tag pointing to the current commit.
17. **git fetch [remote] -** Downloads objects and refs from another repository.
18. **git merge –abort -** Aborts the current conflict resolution process, and tries to reconstruct the pre-merge state.
19. **git rebase [branch] -** Reapplies commits on top of another base tip, often used to integrate changes from one branch onto another cleanly.
20. **git config --global user.name "[name]" and git config --global user.email "[email]" -** Sets the name and email to be used with your commits.
21. **git diff -** Shows changes between commits, commit and working tree, etc.
22. **git remote add [name] [url] -** Adds a new remote repository.
23. **git remote remove [name] -** Removes a remote repository.
24. **git checkout -b [branch] -** Creates a new branch and switches to it.
25. **git branch -d [branch] -**Deletes the specified branch.
26. **git push –tags -**Pushes all tags to the remote repository.
27. **git cherry-pick [commit] -** Picks a commit from another branch and applies it to the current branch.
28. **git fetch –prune -** Prunes remote tracking branches no longer on the remote.
29. **git clean -df -** Removes untracked files and directories from the working directory.
30. **git submodule update --init –recursive -** Initializes and updates submodules recursively.

What is GitHub in DevOps?

GitHub is a cloud-based **Git repository hosting platform** used for **version control** and **collaborative development**. In the context of DevOps, it's more than just a code repository—it becomes a **central hub** for managing **source code**, **automation**, and **collaboration** across development and operations teams.

Why GitHub is Important in DevOps

GitHub is essential in DevOps for version control, enabling teams to track changes, collaborate through branches and pull requests, and maintain code quality. It integrates with CI/CD tools like Jenkins and GitHub Actions to automate builds, tests, and deployments. GitHub also supports Infrastructure as Code (IaC), provides audit logs for compliance, and enables workflow automation.

How GitHub is Used in DevOps

Teams use GitHub to manage code, create branches, and collaborate on development. CI/CD pipelines are triggered via integrations or GitHub Actions. Tasks are tracked using issues and project boards. Code reviews, secrets management, and unified collaboration across Dev, QA, and Ops make GitHub a core DevOps tool.

What is Version Control System?

A Version Control System (VCS) is a system that records changes to a file or set of files over time, so you can recall specific versions later. It allows multiple developers to collaborate on a project by keeping track of changes made to the source code, documents, or any other

DevOps in One Shot

Linux

Linux is a stable, secure, and open-source operating system widely used in DevOps. It supports automation, scalability, and modern tools like Docker, Kubernetes, and Ansible. Its command-line interface and flexibility make it ideal for scripting, customization, and managing deployments across various environments.

Why Linux is Important for DevOps

1. **Open source:** Allows customization and optimization of the OS to fit specific DevOps needs.
2. **Stable & Reliable:** Ideal for critical systems and large-scale deployments, backed by a strong community.
3. **Command-line Interface:** Enables automation, scripting, and efficient system management.
4. **Rich Ecosystem:** Supports essential DevOps tools like Docker, Kubernetes, Ansible, and Puppet.
5. **Compatibility:** Runs across various hardware, distributions, and cloud platforms, ensuring smooth deployments.

Popular Linux Distributions in DevOps

1. **Ubuntu:** User-friendly and widely adopted. Based on Debian, it offers LTS versions with long-term support and strong community backing—great for both beginners and pros.
2. **CentOS:** A stable, enterprise-grade OS based on RHEL. Known for reliability and security in production environments. (*Note: CentOS Stream has replaced traditional CentOS for newer releases.*)
3. **Fedora:** Fast-moving and cutting-edge. Ideal for developers who want the latest features and tech, often serving as a preview of what's coming to RHEL.
4. **Arch Linux:** Lightweight and highly customizable with a rolling release model. Best suited for advanced users who want full control over their environment.
5. **SUSE Linux Enterprise Server (SLES):** Enterprise-focused distribution offering high availability, strong support, and stability—popular in industries like finance and healthcare.

linux Commands That Every DevOps Engineer Should Know

1. **File and Directory Management**

- **ls:** List files and directories.
- **cd:** Change the current directory.
- **pwd**: Print the current working directory.
- **mkdir:** Create a new directory.
- **rm:** Remove files and directories.
- **cp:** Copy files and directories.
- **mv:** Move or rename files and directories.
- **find:** Search for files and directories.
- **chmod:** Change file permissions.
- **chown:** Change file ownership.
- **chgrp:** Change file group ownership.

2. Text Manipulation and Viewing

- **cat:** Concatenate and display file contents.
- **grep:** Search for patterns in files.
- **head:** Display the beginning of a file.
- **tail:** Display the end of a file.
- **less**: View file contents interactively.
- **sed:** Stream editor for text manipulation.
- **awk:** Text processing and data extraction tool.

3. Process and System Management

- **ps**: View running processes.
- **top:** Monitor system resources and processes in real-time.
- **kill:** Terminate processes.
- **systemctl:** Manage system services.
- **service:** Control system services (older Linux distributions).
- **df**: Display disk space usage.
- **du:** Estimate file and directory disk usage.
- **free:** Display system memory usage.
- **uptime:** Show system uptime and load averages.

4. Networking

- **ping:** Send ICMP echo requests to a host.
- **curl or wget:** Download files from the web.
- **ssh:** Securely connect to remote systems.
- **scp:** Securely copy files between systems.
- **netstat:** Network statistics and connections.
- **ifconfig or ip:** Network interface configuration.
- **iptables or ufw:** Firewall configuration.

5. Package Management

- **apt (Debian/Ubuntu) or yum (CentOS/RHEL):** Package management commands for installing, updating, and removing software packages.
- **dpkg (Debian/Ubuntu) or rpm (CentOS/RHEL):** Package management commands for querying package information and managing individual packages.

6. Compression and Archiving

- **tar:** Create and extract tar archives.
- **gzip, gunzip, bzip2, unzip:** Compress and decompress files.
- **zip:** Create and extract ZIP archives.

Package Management Commands

sudo apt update, sudo apt upgrade, sudo apt install <package>, sudo apt remove<package>, apt search <package>, apt show

DevOps in One Shot

Python for DevOps:

Python has gained significant traction in the DevOps ecosystem due to its ease of use, extensive libraries, and adaptability across platforms and tasks. Whether you're automating routine tasks, managing infrastructure, or developing CI/CD pipelines, Python offers a powerful, reliable toolset.

Why Python in DevOps?

Python's popularity in DevOps can be attributed to its simplicity, readability, and powerful libraries, making it ideal for:

1. **Automation**: Python simplifies repetitive tasks, from deployments to monitoring.
2. **Cross-Platform Compatibility**: Scripts written in Python can run on any operating system.
3. **Tool Integration**: Python works with tools like Jenkins, Docker, Kubernetes, and cloud platforms (AWS, GCP, Azure), making it adaptable to a wide range of environments.
4. **Vast Community and Libraries**: Python's extensive package index (PyPI) supports diverse libraries like boto3 for AWS, requests for API interactions, and paramiko for SSH, which greatly enhance DevOps tasks.

These attributes make Python indispensable for DevOps engineers who aim to streamline processes, automate workflows, and manage complex infrastructures efficiently.

Python Scripting Fundamentals for DevOps

Scripting forms the backbone of DevOps automation. Here are some core scripting elements in Python with DevOps applications in mind:

Data Structures and Control Flow

1. **Lists and Dictionaries**: Use lists for ordered data and dictionaries for key-value storage. For instance, a dictionary can store server credentials, and lists can keep track of multiple server IPs.
2. servers = ["10.0.0.1", "10.0.0.2"]
3. server_config = {"hostname": "webserver", "ip": "10.0.0.1", "port": 22}
4. **Loops and Conditionals**: Automate tasks across servers using loops and conditionals.
5. for server in servers:
6. if server == "10.0.0.1":
7. print(f"Connecting to {server}")

Functions

Define reusable functions to modularise tasks:

```
def deploy_application(server, app):
  print(f"Deploying {app} on {server}")
  # Command to deploy
for server in servers:
  deploy_application(server, "nginx")
```

<package>

sudo apt clean **apt –(Advanced Package Tool)**

File I/O

Use Python's file handling to manage configuration files and logs:

```
with open("config.yaml", "r") as config_file:
  config = yaml.safe_load(config_file)
  print(config)
```

These fundamentals help automate and manage tasks more efficiently.

4. Python in CI/CD Pipeline Automation

Python scripts can handle various CI/CD tasks, from building code to managing deployment pipelines.

Automated Builds and Tests

Python's subprocess library enables automating builds and running tests directly from scripts:

```
import subprocess

def build_application():
  subprocess.run(["make", "build"])

def run_tests():
  subprocess.run(["pytest", "tests/"])
```

Integrating with Jenkins and GitHub Actions

Python scripts can interact with CI/CD tools via APIs or command-line utilities:

- **Jenkins API**: Trigger jobs and monitor builds.
- import requests
-
- def trigger_jenkins_job(job_name):
- jenkins_url = f"http://jenkins-server/job/{job_name}/build"
- requests.post(jenkins_url, auth=("user", "password"))
- **GitHub Actions**: Use GitHub API to trigger workflows or monitor statuses.

These scripts allow DevOps engineers to streamline and monitor continuous integration and delivery processes.

Automated Deployment

Deploy applications across environments using paramiko for SSH connections:

```
import paramiko

def deploy_to_server(server, app):
  ssh = paramiko.SSHClient()

ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
  ssh.connect(server, username="admin")
  ssh.exec_command(f"docker run -d {app}")
```

5. Configuration Management with Python

Python can automate configuration management tasks, managing resources across environments.

1. **YAML/JSON Parsing**: Use pyyaml or json for configuration files, common in DevOps for managing application settings.

```
import yaml

with open("app_config.yaml", "r") as f:
    config = yaml.safe_load(f)
```

6. **Configuration Management Tools**: Python can integrate with tools like Ansible or SaltStack for automated configuration changes, ensuring consistency across environments.

6. Infrastructure as Code (IaC) with Python

Python can handle IaC tasks, such as provisioning servers, managing cloud resources, and scaling infrastructure.

Automating AWS Resources with Boto3

boto3 library is essential for AWS resource management.

```
import boto3

ec2 = boto3.resource('ec2')
def create_instance():
  ec2.create_instances(ImageId='ami-12345', MinCount=1, MaxCount=1, InstanceType='t2.micro')
```

IaC scripts enable faster, more reliable infrastructure setups, especially valuable for cloud-native applications.

7. Monitoring and Logging with Python

Python can collect metrics and send alerts when system thresholds are exceeded.

Using Prometheus API for Monitoring

Python can query Prometheus for real-time metrics.

```
import requests

response = requests.get("http://prometheus-server/api/v1/query", params={"query": "up"})
metrics = response.json()
print(metrics)
```

Log Aggregation with Elasticsearch

Use elasticsearch-py for searching and visualising logs:

```
from elasticsearch import Elasticsearch

es = Elasticsearch()
es.index(index="logs", doc_type="log", body={"message": "Error occurred"})
```

Python scripts for automated deployments help maintain consistency across environments.

8. Popular Python Libraries for DevOps

Here are some essential Python libraries for DevOps automation:

- **Boto3**: AWS resource management
- **Requests**: HTTP requests and API interaction
- **Paramiko**: SSH library for secure server communication
- **Docker SDK**: Docker container management
- **Flask**: Lightweight web framework for building monitoring dashboards
- **Prometheus Client**: Collecting and pushing custom metrics to Prometheus

These libraries streamline various DevOps tasks, making automation more accessible and flexible.

9. Best Practices for Using Python in DevOps

To ensure Python scripts are reliable and maintainable, follow these best practices:

- **Use Virtual Environments**: Keep dependencies isolated.
- **Document Code**: Include comments and maintain README files for scripts.
- **Modular Code Structure**: Break tasks into functions for readability.
- **Error Handling**: Implement robust error handling to prevent crashes.
- **Security**: Never hardcode credentials; use environment variables or secrets management.

10. Python DevOps Project Examples

Automated Backup

Create a Python script that archives server logs and uploads them to S3 using boto3.

Deployment Pipeline

Use Jenkins and Python to set up a CI/CD pipeline that automatically tests and deploys new code.

Custom Monitoring Dashboard

A Python-based dashboard using Flask and Prom etheus Client to track application metrics.

11. Conclusion

Python is a versatile tool in DevOps, offering benefits across CI/CD automation, IaC, configuration management, monitoring, and more. By mastering Python, DevOps engineers can enhance productivity, streamline operations, and build resilient, scalable systems.

DevOps in One Shot

Database

Database DevOps integrates database management with DevOps practices to automate and streamline database changes, improving collaboration between development and operations teams. Traditionally, database changes are handled manually, causing bottlenecks in deployment. Database DevOps addresses these challenges by applying version control, automated deployments, and testing to databases, similar to application code.

Key Features of Database DevOps

- **Version Control**: Track and manage database schema changes.
- **Automated Deployment**: Automate database updates across environments.
- **Schema Maintenance**: Ensure consistent schema updates.
- **Rollbacks**: Facilitate easy recovery if issues arise.
- **Release Management**: Streamline the release process for database changes.

Top Challenges in Database DevOps

Data Synchronization: Keeping development and production databases aligned.

Data Security: Protecting sensitive data in non-production environments.

Testing and Validation: Automating database testing procedures.

Schema Evolution: Managing schema changes while ensuring backward compatibility.

Legacy Systems: Integrating Database DevOps with older systems.

Compliance: Ensuring regulatory compliance in database changes.

Performance Tuning: Continuously monitoring and optimizing database performance.

How DevOps Helps Solve These Challenges

- **Automation**: Tools like Ansible, Chef, and Jenkins automate tasks like schema updates and database testing.
- **Version Control**: Git and Liquibase track and manage database changes.
- **Security**: HashiCorp Vault secures sensitive data.
- **Collaboration**: Tools like Slack foster communication between teams.
- **Tooling**: AWS RDS and Docker facilitate database management in DevOps pipelines.

Popular Database DevOps Tools

Open-Source Tools:

- **Liquibase**: Database schema versioning and change management.
- **Flyway**: Database migration tool.
- **Docker**: Containerize databases for better management.

Paid Tools:

- **Redgate SQL Toolbelt**: Tools for database automation.
- **dbForge Studio**: Schema management and version control.
- **AWS Database Migration Service**: Managed database migration and schema conversion.

1. SQL (Structured Query Language) Databases

SQL databases are relational databases that use a structured schema and support SQL to manage and query data. They follow a **tabular structure** with rows and columns.

Key Features of SQL Databases for DevOps:

- **ACID Compliance**: SQL databases typically ensure that database transactions are **Atomic**, **Consistent**, **Isolated**, and **Durable**.
- **Schema-Based**: These databases use predefined schemas that define tables and relationships between them.
- **Transactional Support**: SQL databases are highly reliable for applications that need complex transactions, such as finance and e-commerce.

Popular SQL Databases in DevOps:

2. **MySQL**

Type: Open-source, widely used in web applications.

- **Usage**: MySQL is common for small to medium-sized applications, providing high reliability and speed.

3. **PostgreSQL**
   - **Type**: Open-source, supports advanced features like JSON support, full-text search, etc.
   - **Usage**: Suitable for applications requiring complex queries and high data integrity.
4. **Oracle Database**
   - **Type**: Commercial, widely used in large enterprises.
   - **Usage**: Common in applications that require high availability, security, and support for large datasets.
5. **Microsoft SQL Server**

- **Usage**: Typically used in enterprise environments, especially those built on the Microsoft stack.

SQL Example Use Case in DevOps:

- **Automated Database Migrations**: Tools like **Flyway** and **Liquibase** can be used to automate the process of migrating schemas in SQL databases, ensuring that database changes are consistent across environments (development, staging, production).

2. NoSQL Databases

NoSQL databases are non-relational databases that do not use SQL as their primary query language. They are designed for scalability, flexibility, and performance.

Key Features of NoSQL Databases for DevOps:

- **Schema-less**: NoSQL databases do not require a fixed schema, allowing them to handle unstructured and semi-structured data.
- **Scalability**: Most NoSQL databases are horizontally scalable, which is essential for handling large amounts of data in distributed systems.
- **High Availability**: NoSQL databases typically provide built-in mechanisms for replication and high availability.

Types of NoSQL Databases:

1. **Document Databases**
   - **Example**: **MongoDB**, **CouchDB**
   - **Usage**: Store data as documents (often in JSON format). Suitable for applications that need flexible and hierarchical data storage (e.g., content management systems).
   - **DevOps Integration**: Document databases allow for flexible data storage, making them ideal for applications that evolve rapidly or require handling large amounts of varied data.
2. **Key-Value Stores**
   - **Example**: **Redis**, **Amazon DynamoDB**
   - **Usage**: Store data as key-value pairs, typically used for caching and session management.
   - **DevOps Integration**: Key-value stores can be used in caching layers to speed up response times in high-traffic applications and enable distributed data processing.
3. **Column-family Stores**
   - **Example**: **Apache Cassandra**, **HBase**
   - **Usage**: Store data in columns rather than rows. Suitable for applications requiring fast reads and writes over large datasets (e.g., time-series data).
   - **Type**: Commercial, part of Microsoft's ecosystem.
   - **DevOps Integration**: Used in large-scale applications where performance and scalability are critical, such as real-time analytics and IoT data collection.
4. **Graph Databases**
   - **Example**: **Neo4j**, **Amazon Neptune**
   - **Usage**: Store data in graph structures (nodes, edges, and properties). Ideal for use cases involving relationships and connections (e.g., social networks, recommendation engines).
   - **DevOps Integration**: DevOps teams can integrate graph databases to power applications that require complex querying of interconnected data.

NoSQL Example Use Case in DevOps:

- **Microservices Architectures**: **NoSQL** databases, particularly **Document** and **Key-Value** stores, are widely used in microservices architectures to handle high throughput and flexible data storage needs. For instance, MongoDB is often used in a microservices architecture to manage unstructured data.

Conclusion

SQL and **NoSQL** databases serve different purposes in the DevOps ecosystem, and the choice depends on the needs of the application:

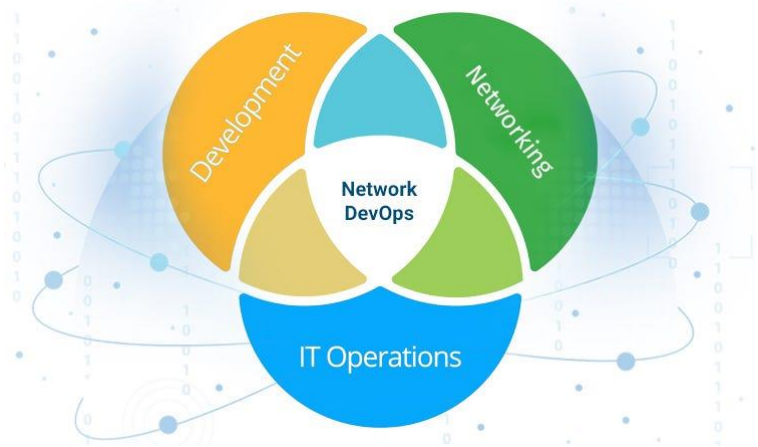
- **SQL databases** are great for applications that require strict data integrity and relational data structures.
- **NoSQL databases** are ideal for modern applications requiring flexible schemas, high scalability, and the ability to handle large volumes of unstructured or semi-structured data.

In DevOps, both types of databases are used based on the architecture, application requirements, and scalability needs. Proper automation, version control, and integration with CI/CD pipelines are essential for managing these databases effectively in a DevOps environment.

For more learn from YouTube

DevOps in One Shot

Networking Fundamentals for DevOps Engineers



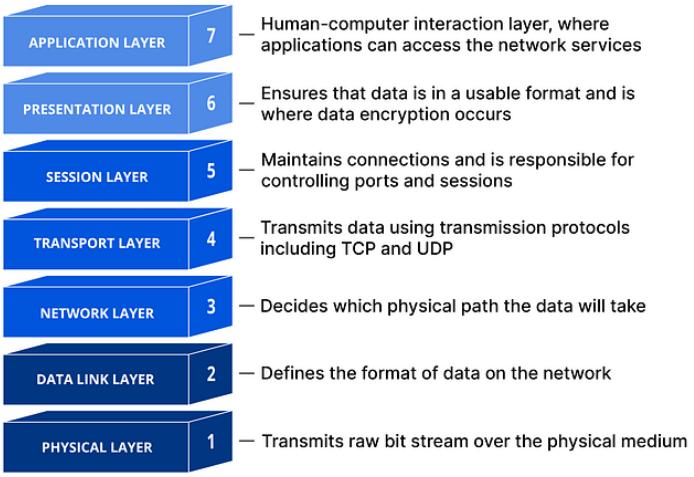
Networking basics are crucial for DevOps engineers as they help understand how system components communicate, troubleshoot issues, secure infrastructure, automate processes, and optimize performance. This knowledge fosters better collaboration between development and operations, especially in distributed applications.

Networking concepts should know:

1. OSI Model
2. Protocols : TCP/UDP/IP
3. Ports
4. Subnetting
5. Routing
6. DNS
7. VPN (Virtual Private Network)
8. Networking tools

1.OSI Model

The OSI (Open Systems Interconnection) model is a conceptual framework that standardizes the functions of a telecommunication or computing system into seven abstraction layers. Each layer has a specific role in managing aspects like hardware, addressing, routing, and application-level interactions.



2. Protocols : TCP/UDP/IP A protocol is a set of rules that defines how data is transmitted and received between devices in a network. It ensures standardized communication, allowing different systems to understand and interact with each other. Examples include TCP/IP, HTTP, and SMTP.

- **TCP (Transmission Control Protocol):** Connection-oriented, ensures reliable and ordered data delivery by breaking data into packets, assigning sequence numbers, and using acknowledgment messages.
- **UDP (User Datagram Protocol):** Connectionless, provides fast, low-latency communication but doesn't guarantee delivery or order, ideal for real-time apps like video streaming.
- **IP (Internet Protocol):** Manages addressing and routing of data packets, ensuring they reach the correct destination. It assigns unique IP addresses and directs data across networks.

3. Ports : Ports are essential for directing network traffic to specific applications or services on devices.

Port Number	Service / Protocol
20	File Transfer Protocol (FTP)
21	File Transfer Protocol (FTP)
22	Secure Shell (SSH) Secure Login
23	Telnet Remote Login (Unsecured)
25	Simple Mail Transfer Protocol (SMTP)
53	Domain Name System (DNS) Service
80	Hypertext Transfer Protocol (HTTP)
110	Post Office Protocol (POP3)
123	Network Time Protocol (NTP)
143	Internet Message Access Protocol (IMAP)
161	Simple Network Management Protocol (SNMP)
443	HTTP Secure (HTTPS)

4. Subnetting is the process of dividing a large network into smaller, more manageable sub-networks, or subnets. This allows for better utilization of IP addresses, improved security, and more efficient routing.
Understanding CIDR (Classless Inter-Domain Routing) notation is key for expressing IP address ranges and subnet masks efficiently. This notation is widely used in network configurations for better organization and clarity.

5.Routing:Routing is the process of directing data packets from a source to a destination across a network. Routers use routing tables and protocols to decide the best path for data transmission, ensuring efficient, reliable communication. Proper routing is essential for network scalability and performance in DevOps.

6. DNS (Domain Name System) DNS is a system that translates human-readable domain names (like www.example.com) into IP addresses that computers can understand. It also helps in locating mail servers, balancing web traffic, redirecting requests, performing reverse lookups, and improving response times through caching. DNS is critical for the reliability and functionality of the internet. **DNS Record Types:** • **A Record (Address Record):** Maps a domain to an IPv4 address. • **AAAA Record:** Maps a domain to an IPv6 address. • **CNAME Record (Canonical Name):** Alias of one domain to another. • **MX Record (Mail Exchange):** Specifies mail servers for a domain. • **NS Record (Name Server):** Identifies authoritative DNS servers for a domain. **7. VPN (Virtual Private Network)** A VPN creates a secure, encrypted connection over the internet, acting like a private tunnel. It allows users to access remote servers, cloud resources, and perform maintenance tasks without compromising security. VPNs are essential for protecting sensitive data and ensuring privacy when managing infrastructure or deployments remotely. **8. Networking Tools** Various networking tools are essential for DevOps engineers to manage, troubleshoot, and optimize network operations. Some important tools include: • **Ping:** Tests connectivity to a host on the network. • **Traceroute:** Displays the route data takes to reach its destination, showing transit delays. • **Netstat:** Shows active network connections, routing tables, and interface statistics. • **Nmap:** Scans a network to discover hosts and services. • **Tcpdump:** Captures and analyzes network traffic for troubleshooting. • **Ipconfig (Windows) / Ifconfig (Linux):** Displays the network configuration. • **Dig:** Queries DNS servers for domain information. • **Nslookup:** Resolves domain names to IP addresses. • **Wireshark:** A protocol analyzer used to capture and analyze network traffic. • **Iperf:** Measures TCP and UDP performance between a server and client.	Notes:

DevOps in One Shot

CI/CDPipeline

A CI/CD pipeline is a series of automated steps that helps software teams deliver code faster, safer, and more reliably. Pipelines are an essential part of continuous integration and continuous delivery (CI/CD) and are critical for modern software development.

Continuous integration (CI) is a process in which developers continually commit code in small increments sometimes multiple times a day which is then automatically built and tested before it is merged with the shared repository.

Continuous delivery (CD) extends CI by automatically deploying all code changes to a QA and/or production environment after the build stage. This ensures that you can release new changes to your customers quickly and sustainably. This practice helps developers automate the deployment process, minimize bugs in production, and increase the speed of software releases.

why CI/CD pipeline is important?

A CI/CD pipeline is important because it automates the process of building, testing, and deploying code, ensuring faster delivery, higher code quality, early bug detection, consistent deployments, and efficient collaboration, which ultimately leads to more reliable and scalable software development.

What is Jenkins?

It's an open-source automation server. It is possible to escalate the software development process with Jenkins automation. Throughout the life cycle, it helps you manage software delivery processes. Various life cycle stages include building, testing, documenting, packaging, staging, deployment, and more.

Why is Jenkins used in DevOps?

Jenkins is a valuable tool for DevOps engineers because it automates software builds, tests, and deployments, enabling continuous integration and delivery (CI/CD), which leads to faster, more reliable software releases.

Features of Jenkins

Jenkins has some features that really sell it as a CI/CD tool. These are some of them:

Plugins Easy to set up Supports most environments Open-source Easy distribution

Plugins

The availability of thousands of plugins allows for a large degree of customizability. The developers have the opportunity to integrate these plugins with development and testing tools, and tailor the architecture to serve a lot of possibilities.

Easy to Set Up

Jenkins has executable packages that are quick to install on Windows, Mac and Unix operating systems. It's just a self-contained Java program that you can set up in ten minutes.

Supports Most Environments

Jenkins supports most environments, including cloud-based architecture. This makes it handy as teams begin to use different environments. It also supports all major cloud platforms, including Amazon, Azure, Google, and IBM Cloud.

Open-source

Jenkins is open-source and free to use. It has an enthusiastic developer community that frequently holds in-person and online meetings.

Easy Distribution

It's easy to distribute Jenkins across work machines, which makes building, testing, and deployment faster.

Security

Jenkins offers security tools to help organizations keep their data and systems safe, right out-of-the-box. This includes features like role-based access control, session management, and encryption. Additionally, Jenkins integrates with several security-related plugins to further enhance its security capabilities.

How Jenkins works?

Jenkins works by automating the software development lifecycle through a pipeline-based approach, where it fetches code from repositories, builds the application, runs tests, manages artifacts, and deploys to production environments, all while integrating with various DevOps tools for monitoring and notifications.

Example : Imagine you're a developer working on an e-commerce website. Every time you write new code or fix a bug, you push your changes to a GitHub repository. Without automation, deploying these changes would require manual testing and deployment, which is time consuming and error-prone. With Jenkins, the entire process is automated. Jenkins automatically detects the change, triggers a pipeline to build, test, store artifacts, deploy the application and notify the team of any failures ensuring fast, reliable, and automated software delivery."

Do it now, don't regret later

DevOps in One Shot

What is GitHub Actions?
GitHub Actions is a CI/CD tool integrated within GitHub, allowing you to automate, customize, and execute your software development workflows directly in your GitHub repository. Whether it's automating tests, deploying to production, or performing complex infrastructure tasks, GitHub Actions has you covered.

2. Why Use GitHub Actions in DevOps?
GitHub Actions has become a cornerstone for modern DevOps pipelines because it:

- Integrates seamlessly with your GitHub repository.
- Automates CI/CD workflows, reducing manual efforts.
- Supports multi-cloud deployment and integrates with popular DevOps tools.
- Provides customizable actions and a marketplace of pre-built actions.
- Supports both self-hosted and cloud-hosted runners, offering flexibility.

3. Core Concepts of GitHub Actions
To fully leverage GitHub Actions, it's important to understand its core components:
Workflows

- **Workflows-** are automated processes defined in .yml files located in the .github/workflows directory of your repository. They are triggered by specific events such as code pushes, pull requests, or scheduled timers (cron jobs).
- **Jobs -** A workflow is made up of one or more jobs. Each job runs in its own runner environment and can execute independently or sequentially, based on dependencies you define.
- **Steps -** Steps are the sequence of tasks executed in a job. Each step can run a script or use an action. All steps in a job run on the same runner, allowing them to share data using the workspace.
- **Actions -** Actions are reusable units of code packaged for use in steps. They can be custom-built or sourced from the GitHub Marketplace. You can also create your own actions using JavaScript or Docker.
- **Runners-** Runners are the servers that execute your workflows. GitHub provides hosted runners for common environments, or you can configure self-hosted runners for custom environments or enhanced control.

GitLab CI/CD for DevOps
GitLab CI/CD (Continuous Integration and Continuous Deployment) is a built-in feature of GitLab that automates the software development lifecycle. It helps DevOps teams build, test, and deploy applications efficiently.

Key Components
.gitlab-ci.yml
This is the core config file stored in the root of your GitLab repository. It defines your entire CI/CD pipeline using stages, jobs, and scripts.

Pipelines
A pipeline is a collection of jobs executed in a defined order (based on stages). It runs automatically when you push code, open a merge request, or trigger it manually.

Stages
Stages define the order in which jobs run (e.g., build, test, deploy). All jobs in a stage run in parallel by default, and the next stage starts only if all jobs in the previous stage succeed.

Jobs
Jobs are the individual tasks (e.g., compile code, run tests, deploy) defined under stages in the .gitlab-ci.yml file. Each job has its own script to execute.

Runners
Runners are agents that execute the jobs. GitLab provides shared runners, or you can configure your own runners (e.g., Docker-based, shell-based) for custom environments.

Why GitLab CI/CD for DevOps?
GitLab CI/CD is essential for DevOps as it automates testing, building, and deployment processes, enhances speed through parallel job execution and caching, ensures security with secret variables and protected branches, offers visibility with real-time pipeline monitoring, and integrates seamlessly with GitLab repositories, issue tracking, and Kubernetes.

Maven is a **build automation tool** primarily used for **Java projects**. It helps manage the build process, dependencies, and project configurations. It simplifies tasks like compiling code, running tests, packaging applications, and deploying artifacts.
Maven automates the process of compiling code, running unit tests, and packaging your application (e.g., .jar, .war files).
mvn clean install

DevOps in One Shot

Containerization

Is the process of packaging an application and all its dependencies into a single, portable, and isolated unit called a container. This container can then be deployed and run consistently across different environments, from development to production, ensuring a smooth and consistent workflow.

Key Aspects of Containerization in DevOps:

- **Packaging and Isolation:**

Containerization bundles an application's code, libraries, runtime environment, and configuration files into a single package. This creates an isolated environment for the application, allowing it to run independently of the host system's operating system.

- **Portability and Consistency:**

Containers are designed to be portable, meaning they can run on any infrastructure that supports containerization technology. This consistency ensures that applications behave predictably regardless of the underlying environment.

- **Automation and Efficiency:**

Containerization streamlines DevOps processes by enabling automation of deployments, scaling, and infrastructure provisioning. This allows developers to deploy applications faster and more efficiently.

- **Microservices Architecture:**

Containerization is well-suited for building and deploying microservices, allowing teams to independently develop, deploy, and scale individual services.

- **CI/CD Integration**:

Containerization seamlessly integrates with CI/CD pipelines, facilitating continuous delivery and deployment.

- **Resource Efficiency:**

Containers are more resource-efficient than traditional virtual machines (VMs), as they share the host operating system kernel.

Benefits of Containerization in DevOps:

- Faster Development Cycles
- Simplified Deployment
- Improved Scalability
- Enhanced Collaboration
- Cost Savings

Docker

Docker is a type of containerization platform that packages your application and all its dependencies together. This ensures that your application works seamlessly in any environment.

Docker is a popular Platform As A Service (PAAS) product. It uses virtualization at the operating system-level and delivers software in packages called containers. These make it easier for a developer to create, run, and deploy applications.

What are Virtual Machines?

A virtual machine is a great imitation of a personal computer. It has its memory, CPU, network interface, and storage to act as an actual physical computer. To implement them you may need special software, hardware, or a mixture.

What is Docker Containers?

It is a unit of code that packages up a new code of an application that the developer is writing and all of its dependencies. It makes the application run faster between different computer settings.

A docker image is a small, standalone, workable package of computer code. It has everything you need to run an application on another system. The components include the code, system tools, runtime environment, system libraries, and settings.

Key Docker Concepts

- **Dockerfile**: A script that defines how to build a container image.
- **Image**: A snapshot of a container. Think of it like a blueprint.
- **Container**: A running instance of an image.
- **Docker Hub**: A public registry where images are stored (like GitHub but for containers).
- **Docker Compose**: A tool to run multiple containers as a single service (like an app with a frontend + backend + DB).

Docker in Action (Simple Flow)

1. You write a Dockerfile describing how to build your app.
2. You build an **image** using docker build.
3. You run a **container** using docker run.
4. You can push the image to **Docker Hub** and let others use it too.

DevOps in One Shot

Setup & Information

- docker --version → Check Docker version
- docker info → Show Docker system information
- docker help → List all Docker commands

Image Commands

- docker build -t <image_name> . → Build an image from a Dockerfile
- docker images → List all local images
- docker rmi <image_id> → Remove an image
- docker pull <image> → Download an image from Docker Hub
- docker push <image_name> → Upload an image to Docker Hub

Container Commands

- docker run <image> → Run a container from an image
- docker run -it <image> → Run a container interactively
- docker run -d <image> → Run a container in detached (background) mode
- docker run -p 8080:80 <image> → Map ports (host:container)
- docker ps → Show running containers
- docker ps -a → Show all containers (including stopped ones)
- docker stop <container_id> → Stop a running container
- docker start <container_id> → Start a stopped container
- docker restart <container_id> → Restart a container
- docker rm <container_id> → Remove a container
- docker logs <container_id> → View container logs
- docker exec -it <container_id> bash → Open a terminal inside a running container

Cleanup Commands

- docker system prune → Remove unused images, containers, and networks
- docker image prune → Remove unused images
- docker container prune → Remove stopped containers

Volumes & Networks

- docker volume ls → List all volumes
- docker volume rm <volume_name> → Remove a volume
- docker network ls → List all networks

Docker Engine

The software that hosts the containers is named Docker Engine. Docker Engine is a client-server based application. The docker engine has 3 main components:

1. Server: It is responsible for creating and managing Docker images, containers, networks, and volumes on the Docker. It is referred to as a daemon process.
2. REST API : It specifies how the applications can interact with the Server and instructs it what to do.
3. Client: The Client is a docker command-line interface (CLI), that allows us to interact with Docker using the docker commands.

Use Cases of Docker

The following are the some of the use cases of Docker:

1. **Continuous Integration and Continuous Deployment (CI/CD)** : It helps in streamlining and automating the software delivery process with ensuring faster and more reliable releases.

Example: A team working on an online store can use Docker to automatically test and update the website every time someone makes a change to the code.

2. **Microservices Architecture:** It facilitates the development, deployment, and management of microservices with enabling independent scaling and maintenance.

Example: A food delivery app can have separate containers for logging in, placing orders, and tracking deliveries. Each part can be updated or fixed without affecting the rest.

3. **Development Environment Consistency:** Docker gives all developers the same environment to work in, no matter what computer they use.

Example: In a team one person is using Windows and another may use Mac, but Docker makes sure the app runs the same way for everyone, avoiding bugs like “it works on my machine and not working in other system”.

4. **Multi-Cloud and Hybrid Cloud Deployments:** Docker makes it simple to run your applications on different cloud platforms like AWS, Azure, or Google Cloud.

Example: A business using AWS today can move their app to Google Cloud tomorrow with little effort, with the help of Docker containers.

Notes:

DevOps in One Shot

Kubernetes

Kubernetes is an open-source container orchestrator that automates the process of deploying, scaling, and operating production apps as highly available, fault-tolerant microservices. It is one of the tools commonly used to achieve DevOps objectives, which simplifies development work by providing a stable foundation for self-service access workflows, CI/CD pipelines, and centralized management of internal platforms.

What are the components of Kubernetes?

Some of the main components of Kubernetes are:

1. **Master**: The master runs the API for the entire cluster of the cloud application.
2. **Nodes**: Nodes are physical or virtual machines within the cluster that host the application.
3. **Pods**: Pods are the basic building blocks of Kubernetes that can run a set of containers simultaneously.
4. **Replication controller**: The replication controller ensures that the expected number of pods are running all the time.
5. **Services**: Services host a dynamic load balancer for a given number of pods

Kubernetes Clusters and Nodes

Kubernetes Clusters: A cluster is a collection of nodes, which are the workhorses that run your applications. Clusters provide the platform where Kubernetes orchestrates container deployment, scaling, and management.

Nodes: These are individual machines or VMs that make up a cluster. Each node hosts a subset of deployed applications managed by the cluster's control plane. Nodes can be physical or virtual and are categorized into master and worker nodes.

Namespaces: These are virtual clusters within a Kubernetes cluster. Use namespaces to help organize resources and provide a scope for names. Namespaces are used to divide cluster resources between multiple users.

Role-Based Access Control (RBAC): RBAC is a method of regulating access to computer or network resources based on the roles of individual users within an organization. In Kubernetes, RBAC is used to control who (or what) can access different resources and operations in a cluster. It's essential for enforcing security policies and ensuring that only authorized users can access specific resources and operations. RBAC is a core element of minimizing the security risks associated with running Kubernetes.

Deployment Strategies in Kubernetes

Deploying applications in Kubernetes can be approached in various ways, each tailored to specific needs and scenarios. Understanding these strategies is crucial for effective management and scalability.

Rolling Updates

Gradually replace the old version of the pod with the new one. This strategy ensures no downtime and is ideal for continuous integration and continuous deployment (CI/CD) processes.

Blue/Green Deployment

Deploy a new version (green) alongside the old version (blue), then switch traffic to the new version after testing. This strategy minimizes risk and downtime.

Canary Deployments

Release a new version to a small subset of users before rolling it out to the entire user base. This approach is useful for testing new features with a fraction of the user base.

Recreate Strategy

All existing pods are killed before new ones are created. This approach may lead to downtime but can be useful for stateful applications that can't handle multiple versions running simultaneously.

Definition

Kubernetes is a container orchestration tool that is used to deploy, scale, and manage containerized applications. Kubernetes follows a master-slave architecture. Here, the master node manages all the Kubernetes cluster operations while the worker nodes deploy and run the containers.

Basic Workflow

1. You define your app's desired state in YAML (e.g. number of pods, container image, ports).
2. You apply it using kubectl apply -f <file>.yaml.
3. Kubernetes continuously works to match the current state to the desired state.

What does kubectl mean?

kubectl stands for Kubernetes Control.

It is the command-line tool used to interact with your Kubernetes cluster.

Think of it like git for GitHub, or docker for Docker. It's the interface to deploy applications, inspect resources, manage cluster components, and much more.

DevOps in One Shot

Commands (using kubectl)

- **kubectl version** is used to check the Kubernetes client and server version.
- **kubectl cluster-info** displays information about the Kubernetes cluster.
- **kubectl get nodes** lists all the nodes in the cluster.
- kubectl get pods shows all the running pods in the current namespace.
- **kubectl get deployments** lists all the deployments in the cluster.
- **kubectl apply -f <filename>.yaml** is used to deploy or update resources defined in a YAML file.
- **kubectl delete -f <filename>.yaml** deletes the resources defined in the specified YAML file.
- **kubectl logs <pod-name>** displays the logs from a specific pod.
- **kubectl exec -it <pod-name> -- bash** opens an interactive shell inside a running pod.
- **kubectl describe pod <pod-name>** shows detailed information about a specific pod, including its status and events.

Kubernetes YAML Example (Deployment + Service) yaml

 # deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: app-container
          image: nginx
          ports:
            - containerPort: 80
```

 # service.yaml

```yaml
apiVersion: v1
kind: Service
  metadata:
  name: my-app-service
spec:
  selector:
    app: my-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
type: LoadBalancer
```

SonarQube in Kubernetes

SonarQube is a code quality and security analysis tool. Running it inside Kubernetes brings all the benefits of container orchestration. Here's why it's a great fit:

- **Scalability –** Easily scale to handle multiple users or CI pipelines.
- **CI/CD Integration –** Seamlessly connects with Jenkins, GitLab, etc., inside your cluster.
- **Cloud-Native Setup –** Centralized, secure deployment using Ingress or LoadBalancer.
- **Resilience –** Pods auto-restart; configs managed via Secrets/ConfigMaps.
- **Resource Control –** Manage CPU/memory for SonarQube and PostgreSQL easily.

Helm is a package manager for Kubernetes, simplifying the deployment and management of applications on Kubernetes clusters. It uses a packaging format called "charts" to bundle Kubernetes resource definitions, making it easier to install, upgrade, and manage applications.

Why use Helm?

Deploying your applications manually seems complex without using Helm. You will have to define every single YAML configuration, from configuring your workloads to how (and what) you want them to be deployed.

Helm is a package manager for Kubernetes, designed to streamline the deployment and management of applications on Kubernetes clusters. It helps you define, install, and upgrade even the most complex Kubernetes applications.

Summary of Helm Components:

1. **Helm Client** – Command-line tool for interacting with Helm.
2. **Helm Charts** – Packages containing Kubernetes resources for an app.
3. **Helm Repositories** – Stores charts, both public and private.
4. **Release** – An instance of a chart deployed in Kubernetes.
5. **Chart Templates** – Templated Kubernetes YAML files for dynamic configuration.
6. **Values Files** – Configuration files for Helm charts.
7. **Tiller** (Helm 2 only) – Server-side component that managed releases (removed in Helm 3).

DevOps in One Shot

Amazon Web Services (AWS) is a comprehensive and widely adopted cloud platform offered by Amazon. It provides a wide range of cloud computing services that help businesses, developers, and organizations build, manage, and scale applications and infrastructure on the cloud.

Group 1: Compute Services

1. **EC2 (Elastic Compute Cloud) —** Provides resizable virtual servers to run applications in the cloud.
2. **Lambda** — Serverless compute service that runs your code in response to events.
3. **Elastic Beanstalk —** Platform-as-a-service (PaaS) for deploying, managing, and scaling applications.
4. **EC2 Container Service (ECS) —** Orchestrates and manages Docker containers.
5. **Fargate —** Serverless compute engine for containers, eliminating the need to manage underlying instances.
6. **Lightsail** — Simplified compute service for deploying and managing applications with a predictable pricing model.
7. **Batch —** Fully managed service for running batch computing workloads.
8. **EC2 Image Builder —** Simplifies the creation and maintenance of custom Amazon Machine Images (AMIs).
9. **Outposts —** Extends AWS infrastructure and services to on-premises environments.
10. **EC2 Spot Instances** — Offers unused EC2 capacity at discounted rates.

Group 2: Storage Services

1. **S3 (Simple Storage Service) —** Scalable object storage for data backup, archiving, and analytics.
2. **EBS (Elastic Block Store) —** Block storage for use with Amazon EC2 instances.
3. **EFS (Elastic File System) —** Managed file storage for use with EC2 instances.
4. **FSx for Windows File Server —** Fully managed Windows-based file system.
5. **FSx for Lustre —** Fully managed high-performance file system for data processing workloads.
6. **Storage Gateway —** Hybrid cloud storage service for on-premises environments.
7. **S3 Glacier —** Low-cost, durable storage for long-term data archiving.
8. **S3 One Zone-Infrequent Access —** Cost-effective storage for infrequently accessed data.

Group 3: Database Services

1. **RDS (Relational Database Service) —** Managed relational database service for MySQL, PostgreSQL, Oracle, SQL Server, and MariaDB.
2. **DynamoDB —** Fully managed NoSQL database service.
3. **ElastiCache —** In-memory caching service for Redis and Memcached.
4. **Neptune —** Fully managed graph database service.
5. **Redshift —** Managed data warehousing and analytics service.
6. **Aurora —** MySQL and PostgreSQL compatible relational database with improved performance and availability.
7. **DocumentDB —** Fully managed, MongoDB-compatible document database.
8. **Timestream —** Serverless, time-series database for IoT and operational applications.
9. **Quantum Ledger Database (QLDB) —** Fully managed ledger database that provides a transparent, immutable, and cryptographically verifiable transaction log.

Group 4: Networking & Content Delivery

1. **VPC (Virtual Private Cloud) —** Isolated virtual network for your AWS resources.
2. **Route 53 —** Scalable and reliable domain name system (DNS) service.
3. **API Gateway —** Fully managed service to create, publish, and maintain APIs.
4. **CloudFront —** Global content delivery network (CDN) service.
5. **Direct Connect —** Dedicated network connection between your on-premises environment and AWS.
6. **Global Accelerator —** Improves the performance of your applications by optimizing the network path.
7. **Elastic Load Balancer (ELB) —** Distributes incoming application traffic across multiple instances for improved availability.

Group 5: Security, Identity & Compliance

1. **IAM (Identity & Access Management) —** Centralized service for managing access to AWS resources.
2. **Cognito —** User identity and authentication management service for web and mobile applications.

3. **GuardDuty —** Continuous security monitoring and threat detection service.
4. **Inspector —** Automated security assessment service for applications deployed on EC2 instances.
5. **Macie —** Uses machine learning to discover, classify, and protect sensitive data in AWS.
6. **Secrets Manager** — Manages secrets like database credentials, on-demand tokens, and API keys.
7. **Security Hub — Centralized security management service for AWS accounts.**
8. **Shield —** Managed DDoS protection service.
9. WAF (Web Application Firewall) — Protects web applications from common exploits and vulnerabilities.
10. **Certificate Manager —** Provisions, manages, and deploys SSL/TLS certificates.

Group 6: Analytics

1. **Athena —** Serverless, interactive query service for analyzing data in Amazon S3.
2. **EMR (Elastic MapReduce) —** Managed big data processing and analysis service using Apache Hadoop and Spark.
3. **Kinesis —** Real-time data streaming and processing service.
4. **QuickSight —** Serverless, scalable business intelligence service for data visualization and analysis.
5. **Redshift Spectrum —** Allows you to run Redshift SQL queries directly on data stored in Amazon S3.
6. **Glue —** Fully managed ETL (Extract, Transform, Load) service for data integration and preparation.

Conclusion

The top 50 AWS services listed above cater to diverse business requirements, ranging from compute, storage, and database management to analytics and security. By understanding these services and their descriptions, you can make informed decisions and select the most appropriate tools to optimize your cloud infrastructure. Keep in mind that AWS constantly innovates and introduces new services, so always stay up-to-date with their offerings to make the most of your cloud experience.

DevOps in One Shot

Azure DevOps is a set of development tools and services provided by Microsoft to help teams plan, develop, test, and deliver software efficiently and collaboratively. It covers the full software development lifecycle (SDLC) and supports DevOps practices such as continuous integration (CI), continuous delivery (CD), and infrastructure as code (IaC).

Here's an overview of Azure DevOps concepts:

Key Components of Azure DevOps:

1. **Azure Boards:**
   - Azure Boards is a tool for managing and tracking your software projects. It includes features for work items, backlogs, sprints, and kanban boards to help teams plan, track, and discuss work.
   - **Work Items:** Represent a unit of work, such as user stories, bugs, or tasks, that need to be completed.
   - **Boards:** Visualize the progress of work, allowing teams to see the status of work items (To Do, In Progress, Done).
2. **Azure Repos:**
   - Azure Repos provides version control to manage your source code. It supports both Git repositories and Team Foundation Version Control (TFVC).
   - **Git Repos**: Distributed version control system for managing code.
   - **TFVC**: Centralized version control for teams that prefer this model.
   - Azure Repos also integrates with tools like pull requests for code reviews and branching strategies for team collaboration.
3. **Azure Pipelines:**
   - Azure Pipelines is a CI/CD service that automates the process of building, testing, and deploying applications. It supports a wide range of programming languages, platforms, and environments.
   - **Continuous Integration (CI):** Automates the process of building and testing code whenever changes are made.
   - **Continuous Delivery (CD):** Automates the release and deployment process, ensuring that code is ready for production at any time.
   - **Multi-Platform Support:** Works with containers, Kubernetes, Windows, Linux, and cloud platforms like Azure and AWS.
4. **Azure Test Plans:**
   - Azure Test Plans is a set of tools for managing tests across your applications. It provides a manual testing environment and enables integration with automated tests.
   - **Test Suites**: Group tests into collections, like functional tests or regression tests.
   - **Automated Testing**: Integrates with test runners like NUnit, MSTest, or Selenium.
5. **Azure Artifacts:**
   a. Azure Artifacts allows teams to host and share packages such as NuGet, npm, Maven, or Python packages in a private repository.
   b. It supports package management and integrates with Azure Pipelines for managing dependencies and building applications.
6. **Azure DevOps Services:**
   a. Azure DevOps is available as both cloud-hosted services and on-premises through Azure DevOps Server (previously known as Team Foundation Server or TFS).
   b. The cloud-based services allow teams to collaborate from anywhere, with scalable infrastructure managed by Microsoft.

Key Concepts of Azure DevOps:

1. **Version Control:**
   - **Git**: A distributed version control system that helps manage code changes, create branches, and collaborate on code with teams.
   - **TFVC**: A centralized version control system that is ideal for large teams and centralized repositories.
2. **CI/CD Pipeline**:
   - **Continuous Integration (CI)**: The process of frequently integrating new code changes into a shared repository, running automated tests, and generating builds to ensure high-quality code.
   - **Continuous Delivery (CD):** The practice of automating the deployment of code to various environments, ensuring that it is always in a deployable state.
3. **Work Item Tracking**:
   - Work items in Azure DevOps represent tasks, user stories, bugs, and other activities to track progress and team collaboration.
   - **Backlog**: A list of work items that need to be completed.
   - **Sprints**: A time-boxed iteration used in agile development where a set of work items are completed.

- **Kanban Boards:** Visualize work progress, with stages like To Do, In Progress, and Done.

4. Agile Project Management:

- Azure DevOps provides tools for Scrum and Kanban methodologies, offering customizable boards, sprints, and reports.
- Epics, Features, and Stories: Hierarchical structure of work items, from larger initiatives (Epics) to smaller tasks (Stories).

5. Release Management:

- Azure DevOps provides a release pipeline that automates the deployment of applications to various environments (development, staging, production).
- **Environment Gates:** Conditional checks (e.g., approvals, manual interventions) to ensure safe deployments.
- **Deployment Strategies:** Supports blue-green deployments, canary releases, and rolling updates.

6. Collaboration and Reporting:

- Azure DevOps integrates with Teams and Slack for team collaboration and notifications.
- **Dashboards and Reports:** Visualize progress, track work items, and monitor pipeline health.
- **Alerts:** Set up email or Slack notifications for pipeline statuses or work item updates.

7. Infrastructure as Code (IaC):

- Azure DevOps supports Infrastructure as Code practices, allowing you to manage your infrastructure (VMs, storage, networks) using code.
- It integrates with Terraform, ARM templates, and Azure CLI to automate the provisioning and management of cloud infrastructure.

Benefits of Azure DevOps:

- End-to-End Lifecycle Management: Azure DevOps supports the full SDLC, from planning and development to testing and deployment.
- Collaboration: Facilitates collaboration between developers, testers, and operations teams, ensuring smoother workflows and faster delivery.
- Automation: Automates build, test, and deployment pipelines, ensuring faster and more consistent releases.
- Scalability: Azure DevOps can scale from small teams to large organizations, supporting both cloud-based and on-premises deployments.
- Integration with Azure and Other Tools: Seamlessly integrates with Azure services and a wide range of third--party tools, making it a powerful addition to any tech stack.

Conclusion:

Azure DevOps provides a suite of powerful tools to support the DevOps culture, helping teams automate their workflows, improve collaboration, and increase productivity. With services ranging from version control and CI/CD pipelines to agile project management and testing, Azure DevOps makes it easier to manage and deliver high-quality software. Whether you are building on Azure or other cloud platforms, Azure DevOps offers the flexibility and scalability needed for modern software development.

DevOps in One Shot

GCP

Google Cloud Platform (GCP) is a suite of cloud computing services provided by Google. It offers infrastructure as a service (IaaS), platform as a service (PaaS), and serverless computing environments. GCP enables businesses to build, deploy, and scale applications and services using the same infrastructure Google uses internally for products like Google Search, YouTube, and Gmail.

Core Concepts & Services of GCP

1. Compute Services

- **Compute Engine:** Scalable virtual machines (VMs) on Google infrastructure.
- **App Engine:** Platform for building scalable web apps and mobile backends with no server management.
- **Cloud Functions:** Lightweight, event-driven serverless functions (FaaS).
- **Cloud Run:** Deploy containers easily without managing servers.
- **GKE (Google Kubernetes Engine):** Managed Kubernetes service for containerized applications.

2. Storage & Databases

- **Cloud Storage:** Object storage for unstructured data like media files, backups, and archives.
- **Cloud SQL:** Managed relational databases (MySQL, PostgreSQL, SQL Server).
- **Cloud Spanner:** Globally distributed relational database with high availability and horizontal scalability.
- **Firestore / Datastore:** NoSQL document database for web and mobile applications.
- **Bigtable:** Scalable NoSQL wide-column database for big data workloads.
- **Filestore:** High-performance file storage for applications that require a file system interface.

3. Networking

- **VPC (Virtual Private Cloud):** Private network space for your GCP resources.
- **Cloud Load Balancing:** Distributes traffic across multiple instances and regions.
- **Cloud CDN:** Content delivery network to serve content closer to users globally.
- **Cloud Interconnect:** Private, high-bandwidth connection between your infrastructure and GCP.
- **Cloud NAT:** Enables outbound internet access for instances without external IP addresses.

4. Identity & Security

- **Cloud IAM (Identity and Access Management):** Manage access to GCP resources.
- **Cloud Identity:** Identity as a service for user management and authentication.
- **KMS (Key Management Service):** Create and manage encryption keys.
- **Cloud Armor:** DDoS protection and web application firewall (WAF).
- **Security Command Center:** Security and risk platform to detect misconfigurations and threats.

5. Developer Tools & DevOps

- **Cloud Build:** CI/CD platform to build, test, and deploy code across all GCP services.
- **Artifact Registry:** Store and manage container images and language packages.
- **Cloud Source Repositories:** Git repositories hosted on Google Cloud.
- **Operations Suite (formerly Stackdriver):** Monitoring, logging, and diagnostics for apps on GCP and AWS.

6. Data & Analytics

- **BigQuery:** Fully managed, serverless data warehouse for real-time analytics.
- **Dataflow:** Stream and batch data processing with Apache Beam.
- **Dataproc:** Managed Apache Hadoop and Spark for big data processing.
- **Pub/Sub:** Asynchronous messaging service for event-driven systems.
- **Looker:** Business intelligence and data exploration platform.

7. AI & Machine Learning

- **Vertex AI:** Unified AI platform for building, training, and deploying ML models.
- **AutoML:** Train high-quality custom models with minimal effort.
- **AI APIs:** Pre-trained APIs for vision, speech, language, and translation.

8. Hybrid & Multi-Cloud

- **Anthos:** Manage applications across on-prem, GCP, and other clouds with consistent policies and service mesh.
- **Migrate to GCP:** Tools and services for migrating VMs and workloads from other platforms.

Do it now, don't regret later

Benefits of GCP

- Performance & Scalability: Built on Google's high-speed network.
- Security: Industry-leading security model, same used by Google products.
- Flexible Pricing: Pay-as-you-go, sustained use discounts, and preemptible VMs.
- Hybrid-Cloud Friendly: Seamless tools for hybrid and multi-cloud with Anthos.
- Strong Analytics & AI Integration: Leading tools for big data and AI/ML workflows.

notes:

Infrastructure as Code (IaC)

Infrastructure as Code (IaC) is a method of managing and provisioning IT infrastructure using code rather than manual configuration. It allows teams to automate the setup and management of their infrastructure, making it more efficient and consistent. This is particularly useful in the DevOps environment, where teams constantly update and deploy software.

Working of Infrastructure as Code (IaC)

Infrastructure as Code is a method of defining and managing your system's infrastructure using code. Just like software code determines the logic and functionality of an application, IaC describes the architecture of the system, including components such as servers, networks, storage, and operating systems.

Role of IaC in DevOps

DevOps focuses on ensuring that development and operations teams deliver software more quickly and with higher quality. Infrastructure as Code plays a strong role in this, automating the management of infrastructure along with continuous integration and continuous delivery pipelines.

Features of IaC

- **Automation:** IAC automates the provisioning and configuration of infrastructure, reducing manual errors and saving time.
- **Repeatability:** IAC scripts can be used repeatedly, making it easy to recreate the same infrastructure in multiple environments.
- **Version Control:** IAC code is stored in version control systems like Git, which makes it easy to track changes, revert to previous versions, and collaborate with others.
- **Scalability:** IAC makes it easy to scale infrastructure up or down, adding or removing resources as needed.
- **Transparency:** IAC makes the infrastructure transparent and understandable, as the code defines the infrastructure components and their relationships.
- **Improved Security:** IAC helps ensure that infrastructure is configured consistently and securely, reducing the risk of security vulnerabilities.

Applications of IaC

Infrastructure as Code has a wide range of applications across different domains, including

- **Cloud computing:** IAC is widely used in cloud computing, where it can be used to provision and configure cloud resources, such as virtual machines, storage, and databases.

- **DevOps:** IAC is a key component of DevOps, where it is used to automate the deployment and management of infrastructure and applications.
- **Continuous integration and delivery (CI/CD):** IAC is used in CI/CD pipelines to automate the deployment and configuration of infrastructure and applications.
- **Networking**: IAC can be used to automate the deployment and management of networks, including creating and managing subnets, security groups, and firewalls.
- **Web application deployment:** IAC can be used to automate the deployment and management of web applications, including specifying the web server, application server, and load balancer.
- **Database deployment:** IAC can be used to automate the deployment and management of databases, including specifying the database engine, creating tables, and configuring users.
- **Big data:** IAC can be used to automate the deployment and management of big data infrastructure, including setting up clusters and configuring data processing frameworks such as Apache Hadoop or Apache Spark.

Difference between declarative and imperative IaC

- **Declarative approach**

By adopting this approach, you can define the required state of your system and include the resources and characteristics it should possess. An IaC tool takes care of how to implement it and achieve the desired results on its own. IaC tools like Terraform, AWS CloudFormation, Ansible, and Puppet are preferred among the developer community.

- **Imperative approach**

This approach requires you to list the steps the IaC tool should follow while providing a resource. These series of command imperatives instruct the tool to build each environment. A popular example of an imperative IaC tool is Chef. Although IaC tools are generally compatible with both approaches, they prefer a particular approach over the other.

Top tools to implement Infrastructure as Code

AWS CloudFormation

With the help of CloudFormation, you can easily and quickly provision and manage different AWS and third-party resources with IaC.

Key features include:-

- Scales your resources and automates resource management by integrating with different AWS resources.
- Leverages open-source CLi to build resource providers for provisioning and managing third-party application resources.
- Codes your infrastructure from scratch using a preferred template language, while CloudFormation provisions and manages the **tech stack** and resources described in the template.

Puppet

It's an open-source configuration management tool. Puppet's main function is to manage multiple application servers simultaneously.

Key features include:-

- Ruby-based Domain Specific Language (DSL) allows you to describe your infrastructure's necessary end state.
- Reduction in downtime due to misconfiguration and effective disaster recovery.
- Supports Mac OS, Microsoft Windows, Debian, and more with easy-to-learn language for defining configurations.

Ansible

Red Hat introduced Ansible intending to promote simplicity in automation. From provision and configuration to application management, it enables automating all the processes with ease.

Key features include:-

- Creates multiple identical environments with all the security baselines while Ansible takes care of the compliance requirements.
- Executes playbooks to create and manage the necessary infrastructure resources.
- Code written in YAML enables understanding and deploying the configurations easily. Also, you can expand its features to write your own Ansible modules and plugins.

Terraform

A popular and open-source tool for infrastructure automation, Terraform is used for provisioning, configuring, and managing the infrastructure code.

Key features include:-

- Utilizes the same CLi workflow to plan and build IaC across different infrastructure providers.

Provision of multiple environments with the same configuration to manage the complete

- lifecycle of your infrastructure. It significantly reduces human errors and promotes maximum automation.
- Pre-execution check allows configuration validation to ensure that the configuration meets the expected

results before updating/provisioning the infrastructure.

Chef

A developer's favorite, Chef focuses on deploying and modeling a scalable and secure automation process across any environment.

Key features include:-

- Uses procedural style language, so users can write the code and describe step-by-step how to achieve the required state. Also, the user can choose the optimal deployment process.
- Uses Ruby-based DSL to create recipes and cookbooks to specify the definite steps for achieving the application configuration.
- Enables DevOps teams to provision and deploy on-demand infrastructure easily.

SaltStack

It is an open-source IaC tool that uses a master-slave architecture to manage infrastructure. It is known for its high-speed communication and efficient data processing capabilities.

Vagrant

It is a tool for managing virtual development environments using IaC. It allows developers to define their development environments as code and share it with others.

Packer

It is a tool for creating machine images for multiple platforms using IaC. It allows for consistent and repeatable image creation across various environments.

Terraform

Terraform is an **open-source Infrastructure as Code (IaC)** tool created by HashiCorp. It lets you **define, provision, and manage cloud infrastructure** using code. It's **cloud-agnostic**, so you can manage resources on AWS, Azure, GCP, and many more.

Terraform in a DevOps Workflow

1. **Write –** Develop infrastructure as code (IaC) configuration to define the infrastructure to be provisioned and managed.
2. **Plan –** Preview the modifications necessary before implementation as a means to verify changes before applying them to the infrastructure environment.
3. **Apply –** Reliably and scalably apply infrastructure changes in an automated fashion to ensure continuity across environments.

Why is Terraform Important for DevOps?

DevOps is all about **automation**, **repeatability**, and **collaboration**. Terraform helps you:

- Eliminate manual configurations
- Version control your infrastructure (just like code!)
- Reproduce environments easily
- Integrate with CI/CD pipelines
- Prevent drift and maintain consistency

Terraform Project Structure

```
terraform/
├── dev/
│   └── main.tf
├── prod/
│   └── main.tf
├── modules/
│   └── ec2/
│   └── main.tf
├── variables.tf
├── outputs.tf
└── backend.tf
```

DevOps in One Shot

Terraform Commands

1. **terraform init**
   Initializes the working directory by downloading the provider plugins and setting up the backend for state management.
2. **terraform validate**
   Checks the configuration files for syntax errors and validates them against the provider schema.
3. **terraform fmt**
   Automatically formats Terraform configuration files to follow standard style conventions.
4. **terraform plan**
   Creates an execution plan, showing what actions Terraform will take to reach the desired infrastructure state.
5. **terraform apply**
   Applies the changes required to achieve the desired infrastructure, as defined in the configuration files.
6. **terraform destroy**
   Removes all resources defined in the Terraform configuration, effectively tearing down the infrastructure.
7. **terraform output**
   Displays the values of outputs defined in the configuration, useful for referencing key data like IPs or URLs.
8. **terraform show**
   Provides a human-readable output of the current state of infrastructure managed by Terraform.
9. **terraform state**
   Allows advanced manipulation and inspection of Terraform's state file, including listing and showing resources.
10. **terraform taint**
    Manually marks a specific resource for recreation during the next apply, useful for forcing updates.
11. **terraform untaint**
    Removes the tainted flag from a resource, preventing it from being recreated.
12. **terraform workspace**
    Manages multiple workspaces, allowing you to use the same configuration across environments like dev, staging, and production.
13. **terraform login**
    Authenticates the local Terraform CLI with Terraform Cloud.
14. **terraform test**
    Executes tests written in HCL for validating Terraform configurations (available in Terraform 1.6+).

Core Elements of Terraform

1. **Terraform CLI**
   The command-line interface used to manage infrastructure. Key commands include:
   - init: Initializes the working directory
   - validate: Checks syntax
   - plan: Previews changes
   - apply: Applies changes
   - destroy: Deletes infrastructure
2. **Terraform Language (HCL)**
   Uses **HashiCorp Configuration Language (HCL)** to define infrastructure as code. Resources like VMs, VPCs, or buckets are written in human-readable blocks.
3. **Terraform Providers**
   Plugins that let Terraform interact with platforms like AWS, Azure, GCP, etc. They define the resources Terraform can manage.
4. **Terraform Modules**
   Reusable sets of Terraform configurations that organize and simplify infrastructure code. Modules support input/output variables and can be nested or shared via the Terraform Registry.
5. **Terraform Provisioners**
   Used to run scripts or commands on resources after creation. Useful for bootstrapping but should be used sparingly.
6. **Terraform State**
   A file (terraform.tfstate) that tracks current infrastructure. Can be stored locally or remotely (e.g., S3) for team collaboration.

Do it now, don't regret later

DevOps in One Shot

ANSIBLE

Ansible

Ansible is an open-source automation and configuration management tool that simplifies IT infrastructure management. It automates tasks like provisioning, configuration, application deployment, and orchestration, making it a popular choice for DevOps and systems administrators.

Ansible Architecture

1. **Control node:** Commands and Playbooks can run by invoking /usr/bin/ansible or /usr/bin/ansible-playbook, from any control node. You can use any computer that has Python installed on it as a control node. However, one can not use a computer with Windows OS as a control node. One can have multiple control nodes.
2. **Managed nodes:** Also sometimes called "hosts", Managed nodes are the network devices (and/or servers) you manage with Ansible.
3. **Inventory:** Also sometimes called "hostfile", Inventory is the list of Managed nodes use to organize them. It is also used for creating and nesting groups for easier scaling.
4. **Modules:** These are the units of code executed by Ansible. Each module can be used for a specific purpose. One can invoke a single module with a task, or invoke several different modules in a playbook.
5. **Tasks:** The units of action in Ansible. One can execute a single task once with an ad-hoc command.

Ansible Playbook With Example

These are the ordered list of tasks that are saved so you can run those tasks in that order repeatedly. Playbooks are written in YAML and are easy to read, write, share and understand.Ansible playbooks can perform wide variety of tasks as mentioned below

- Deploying and configuring applications
- Managing system configurations
- Orchestrating complex workflows

Running Playbooks

6. ansible-playbook site.yml -i inventory.ini — Run a playbook.
7. ansible-playbook site.yml --syntax-check — Validate playbook syntax.
8. ansible-playbook site.yml --check — Perform a dry run.
9. ansible-playbook site.yml -K — Prompt for sudo password.
10. --private-key ~/.ssh/id_rsa — Use a custom SSH private key.

Example

The language used to write the ansible playbooks was YAML which is human redable.Following sections will consists in ansible playbook.

```
---
- hosts: all
  tasks:
    - name: Install the Apache web server
      apt:
        name: apache2
        state: present
```

1. **Hosts:** Host in ansible play book defines no.of remote host that playbook will executed and performs the task specified.
2. **Vars:** Vars in anisble playbook defines the variables which can be used through out the playbook.
3. **Tasks:** This is the section where we are going tot mention the type of task to be executed in the host servers.

Types of Ansible Modules

Following are the some modules which are frequently used in the ansible.

1. System modules
2. Application modules
3. Cloud modules
4. Networking modules
5. Command modules

Essential Ansible Commands

Inventory & Setup

1. ansible --version — Check the installed version of Ansible.
2. ansible all -m ping -i inventory.ini — Verify connectivity with all hosts.

Ad-Hoc Commands

3. ansible all -m shell -a "uptime" — Run a shell command on all hosts.
4. ansible all -m copy -a "src=./file.txt dest=/tmp/" — Copy a file to remote hosts.
5. ansible all -m apt -a "name=nginx state=present" — Install a package (Debian).

Do it now, don't regret later

DevOps in One Shot

Monitoring and Logging

1. Monitoring

Definition: Monitoring refers to the continuous tracking of system performance and health. It involves observing the various metrics that describe the system's functioning, ensuring it's running as expected.

Key Aspects of Monitoring:

- **Metrics**: Quantitative data, such as CPU usage, memory consumption, response times, and network throughput.
- **Alerts**: Notifications triggered by specific thresholds or anomalous behaviors (e.g., high CPU utilization).
- **Dashboards**: Visual representations of system health and performance metrics, typically updated in real time.

Purpose:

Monitoring helps proactively detect issues before they escalate into failures, providing insight into system performance and ensuring availability.

Tools:

- **Prometheus**: For real-time monitoring and alerting of metrics.
- **Nagios**: For infrastructure monitoring and alerting.
- **Zabbix**: For comprehensive monitoring of networks, servers, and applications.
- **Datadog**: For cloud-scale monitoring with dashboards and alerting.
- **Grafana**: For creating dashboards based on data from Prometheus or other sources.

Example:

- Monitoring CPU usage of a server to alert when it crosses 80% usage, which can trigger scaling actions in cloud environments.

2. Logging

Definition: Logging refers to the process of recording events, activities, and states that occur within a system or application. Logs contain detailed information about system behavior, errors, transactions, and more.

Key Aspects of Logging:

- **Log Levels**: Categorization of logs, such as:
  - **INFO**: General operational information.
  - **DEBUG**: Detailed information for debugging.
  - **ERROR**: Issues that cause problems, usually requiring immediate attention.
  - **WARN**: Potentially problematic events, but not critical.
- **Log Aggregation**: Collecting logs from multiple sources (e.g., servers, containers, apps) into a central system.
- **Log Search**: The ability to query logs for troubleshooting or analysis.

Purpose:

Logging helps in diagnosing issues, performing audits, and understanding what happened during specific incidents. It provides historical insights into system behavior.

Tools:

- **ELK Stack (Elasticsearch, Logstash, Kibana)**: For storing, processing, and visualizing log data.
- **Splunk**: For analyzing machine data and logs in real time.
- **Fluentd**: For log aggregation and forwarding to various backends.
- **AWS CloudWatch Logs**: For logging and monitoring within the AWS ecosystem.
- **Graylog**: An open-source tool for managing and analyzing log data.

Example:

- An application generates error logs when a database connection fails, and these logs can be searched and analyzed to determine the root cause of the failure.

Integration into the DevOps Workflow:

Monitoring and logging are seamlessly integrated into the DevOps workflow, ensuring continuous feedback and facilitating the cycle of continuous integration, delivery, and deployment. Let's examine their placement within the typical DevOps flow:

- **Development**: Developers instrument their code with appropriate logging statements to capture relevant information during runtime. These logs can help track application behavior, identify code issues, and enable efficient debugging during the development phase.
- **Continuous Integration (CI):** Monitoring and logging tools can be integrated into the CI process to capture build and test metrics, providing visibility into code quality and identifying potential integration issues.
- **Continuous Delivery (CD):** Monitoring and logging play a vital role during CD by capturing application performance and infrastructure health metrics in pre-production and production environments. They help verify the successful deployment of new features or changes and ensure the system is functioning as expected.

- **Production Environment:** In the live production environment, monitoring tools continuously collect metrics on system health, response times, resource utilization, and more. Alerts and notifications can be configured to notify the appropriate teams in case of anomalies or critical incidents.

Prometheus: Prometheus is a widely adopted open-source monitoring solution known for its scalability, flexibility, and strong integration capabilities. It follows a pull-based model, where it scrapes metrics from target systems at regular intervals. Prometheus stores data as time-series, allowing for efficient querying, alerting, and visualization. It excels in dynamic environments and is particularly well-suited for containerized applications and microservice architectures. With its robust ecosystem of exporters and integrations, Prometheus can monitor a diverse range of systems and applications.

Grafana: Grafana is a popular open-source visualization platform that integrates seamlessly with various data sources, including Prometheus. It allows users to create rich, interactive dashboards and graphs, providing real-time insights into system performance and metrics. Grafana's extensive plugin library enables integration with numerous data sources and offers a wide range of visualization options. It supports alerting and notification features, empowering teams to respond proactively to critical events. Grafana's intuitive interface and user-friendly features make it a go-to choice for visualizing and exploring data from monitoring and logging tools.

ELK Stack (Elasticsearch, Logstash, Kibana): The ELK Stack, now commonly referred to as the Elastic Stack, is a powerful combination of open-source tools for log management and analysis. It consists of Elasticsearch, a distributed search and analytics engine; Logstash, a versatile data processing pipeline; and Kibana, a web-based visualization platform. The Elastic Stack enables centralized collection, storage, and analysis of logs from various sources. Logstash allows for log ingestion, parsing, and transformation before sending the data to Elasticsearch for indexing and storage. Kibana provides a user-friendly interface to search, analyze, and visualize log data effectively. The Elastic Stack is highly scalable, well-suited for large-scale log processing, and offers advanced search and analysis capabilities.

Datadog: Datadog is a popular cloud-based monitoring and analytics platform that provides comprehensive observability solutions. It offers a wide range of monitoring capabilities, including infrastructure monitoring, application performance monitoring (APM), log management, and synthetic monitoring. With a unified platform and a rich set of integrations, Datadog enables teams to monitor and correlate metrics, traces, and logs from different systems and services. It provides intuitive dashboards, sophisticated alerting mechanisms, and extensive data analysis features. Datadog's cloud-native architecture and ease of use make it a preferred choice for teams seeking a robust and all-encompassing monitoring solution.

Splunk

Splunk is a powerful platform designed for searching, monitoring, and analyzing machine-generated data. It can handle data from various sources, including logs, events, metrics, and more. Splunk provides real-time insights into operational data, enabling organizations to gain a comprehensive understanding of their systems.

DevOps in One Shot

Introduction

In the world of DevOps, automation is key to managing complex systems, streamlining workflows, and accelerating development cycles. **Shell scripting** is a versatile and powerful tool that allows DevOps engineers to automate routine tasks, reducing manual effort and ensuring reliability across processes. In this guide, we will cover the essentials of DevOps automation with shell scripting, exploring various use cases, techniques, and best practices.

2. Why Automate with Shell Scripting?

Shell scripts are widely used for automation because:

- They're **lightweight and quick to execute** on almost any system with a shell (like Bash on Linux).
- They can **integrate with other automation tools** in DevOps, like Jenkins, Ansible, and Kubernetes.
- Shell scripting is often faster to implement for simple automation tasks compared to heavier programming languages.

3. Basic Concepts of Shell Scripting

Before diving into specific use cases, let's review some basics of shell scripting that are essential for any DevOps automation project.

3.1 Essential Shell Commands

Some fundamental shell commands you should be familiar with include:

- echo: Print text to the terminal.
- cat, tail, head: Work with file content.
- grep: Search for patterns in files.
- awk, sed: For text processing.
- curl, wget: Fetch data from the web.

Example:

```
echo "Hello, DevOps!"
cat /etc/passwd | grep "user_name"
```

3.2 Variables and Data Types

Shell scripting has basic variable support. Declare variables as follows:

```
NAME="DevOpsEngineer"
echo "Hello, $NAME"
```

- Variables are created without explicit types, but you can use declare for specific types (like -i for integers).
- Use export to make a variable accessible to subprocesses.

3.3 Conditionals

Use if, elif, and else to create conditional statements.

```
if [ $USER == "devops" ]; then
  echo "Welcome, DevOps Engineer!"
else
  echo "Access denied."
fi
```

- == and != are used for string comparison.
- -eq, -ne, -gt, -lt, etc., are used for integer comparisons.

3.4 Loops

Loops are essential for automating repetitive tasks.

- for loop:

```
for i in {1..5}; do
  echo "Number $i"
done
```

- while loop:

```
count=1
while [ $count -le 5 ]; do
  echo "Count $count"
  ((count++))
done
```

4. Setting Up Your Environment for Automation

Ensure you have a suitable environment:

- **Shell**: Typically, Bash (/bin/bash) on Linux.
- **Text Editor**: Use Vim, Nano, or any preferred editor for writing scripts.
- **Permissions**: Grant executable permissions to scripts using chmod +x script.sh.

5. Shell Scripting for DevOps Use Cases

Now, let's go over specific DevOps tasks that can be automated using shell scripting.

5.1 System Monitoring

Monitoring system resources like CPU, memory, and disk usage can be automated to alert the DevOps team when resources exceed thresholds.

Example script for CPU usage monitoring:

```
#!/bin/bash
THRESHOLD=80
cpu_usage=$(top -bn1 | grep "Cpu(s)" | sed "s/.*, *\([0-9.]*\)%* id.*/\1/" | awk '{print 100 - $1}')
if (( $(echo "$cpu_usage > $THRESHOLD" |bc -l) )); then
  echo "CPU usage is above $THRESHOLD%. Current usage: $cpu_usage%" | mail -s "CPU Alert" you@example.com
fi
```

5.2 Automating Deployments

Shell scripts can simplify deployments by pulling the latest code, building, and deploying it to a server.

Example deployment script:

```
#!/bin/bash
echo "Starting deployment..."
git pull origin main
docker build -t myapp:latest .
docker stop myapp || true
docker rm myapp || true
docker run -d --name myapp -p 80:80 myapp:latest
echo "Deployment completed."
```

5.3 Backup and Restore

Regularly backing up data is crucial. Shell scripts can automate backup tasks and store them in secure locations.
Example for MySQL database backup:

```
#!/bin/bash
DATE=$(date +%F)
DB_BACKUP_PATH='/backup/dbbackup'
DB_USER='root'
DB_PASS='password'
DB_NAME='mydb'

mkdir -p ${DB_BACKUP_PATH}/${DATE}
mysqldump -u${DB_USER} -p${DB_PASS} ${DB_NAME} >
${DB_BACKUP_PATH}/${DATE}/${DB_NAME}.sql
echo "Backup completed for database ${DB_NAME}."
```

5.4 Log Management

Log files can grow large over time. Shell scripts can archive, rotate, or delete old logs to save space.
Example for archiving logs:

```
#!/bin/bash
LOG_DIR="/var/log/myapp/"
ARCHIVE_DIR="/var/log/myapp/archive/"
find $LOG_DIR -name "*.log" -mtime +30 -exec mv {}
$ARCHIVE_DIR \;
gzip $ARCHIVE_DIR/*.log
echo "Log files older than 30 days archived and
compressed."
```

6. Advanced Shell Scripting Techniques

Advanced shell scripting can make scripts more powerful and adaptable. Here are a few techniques:

- **Functions**: Break scripts into reusable functions.

```
function backup() {
  echo "Performing backup..."
  # Backup logic here
}
backup
```

- **Error Handling**: Use set -e at the beginning to exit on errors or use traps to handle errors gracefully.
- **Parsing Arguments**: Allow flexibility by parsing command-line arguments.

```
while getopts u:p: flag
do
  case "${flag}" in
    u) user=${OPTARG};;
    p) pass=${OPTARG};;
  esac
done
echo "User: $user"
echo "Password: $pass"
```

7. Best Practices in Shell Scripting for DevOps

Following best practices will improve the readability, reliability, and performance of your scripts:

- **Use Comments**: Comment your code for clarity.
- **Use Descriptive Variable Names**: Improve readability.
- **Modularize with Functions**: Keep code organized by using functions.
- **Error Handling**: Handle potential errors with conditions or traps.
- **Limit Root Access**: Run scripts with the least privilege necessary.

8. Hands-On Projects for Practice

Here are some hands-on projects to help you practice and improve your DevOps shell scripting skills:

1. **Automated Server Provisioning**: Write a script that sets up a web server (e.g., Apache or Nginx) on a new Linux instance.
2. **CI/CD Pipeline Script**: Write a shell script to automate steps of a CI/CD pipeline (e.g., cloning, building, and deploying).
3. **User Access Management**: Automate user creation, permission setting, and access logging.
4. **Automated Health Check**: Create a script that checks the health of critical services and sends alerts if services are down.